

Student Attendance and Leave Approval System

Screenshots of every screen of the user interface: insert, delete and view, before and after, with the matching MySQL output.

Group members

Shaurya Mittal · 25WU0101128

Vanshika Rao · 25WU0701027

Samarth Samanekar · 25WU0101119

How to read this document

The screens follow the application in the order it is used. Each change is shown before and after, and the SQL sent to MySQL is visible in the panel under the table. Where a change was made, the MySQL command line output underneath shows the same data read back from the database, so every UI change can be matched with the database.

Technology: Flask web app (Python) with PyMySQL, connected to a MySQL-compatible server (MariaDB 10.11) running the database attendance_leave. The screenshots were captured from the running application.

Contents

Application screens	3
1. View: attendance before any leave is approved	3
2. Insert: a new leave request (student Rohit Sharma)	4
3. Delete: cancel a pending request	9
4. Two-level approval and its effect on attendance	11
5. Insert and delete a record: department	17
6. Insert and delete a record: student	25
7. Attendance: marking and the report	31
8. Database tab: what is stored in MySQL	34

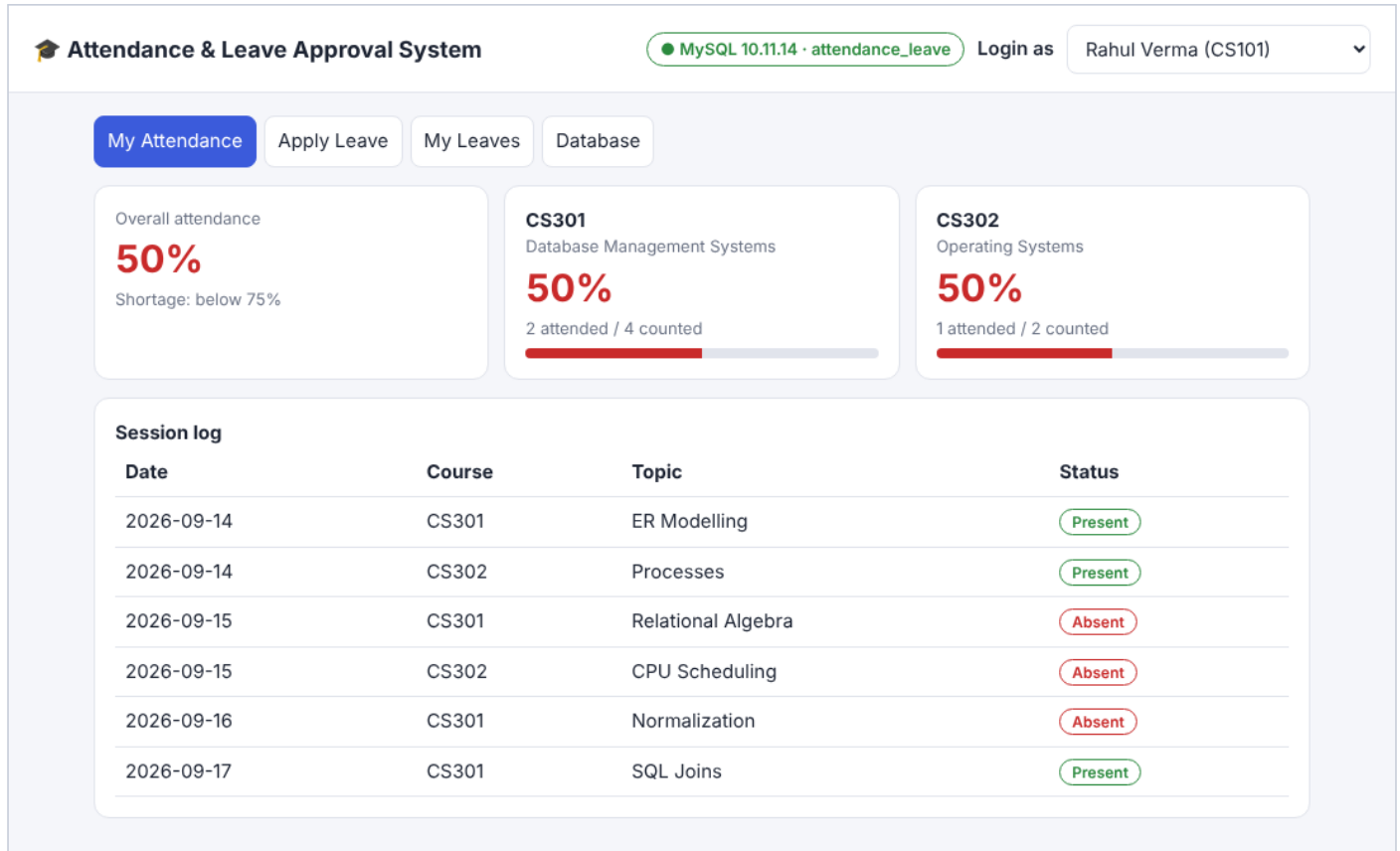
Application screens

Insert, delete and view, each shown before and after.

1. View: attendance before any leave is approved

VIEW Figure 1. My Attendance, student Rahul Verma (before approval)

Overall attendance is 50%, below the 75% requirement, so the shortage is flagged. The session log shows three sessions marked Absent on 15 and 16 September.



MYSQL MySQL output for Figure 1

Rahul's attendance before approval: 50% in both courses.

```
MySQL command line: database attendance_leave

mysql> SELECT roll_no, student_name, course_code, attended, total_sessions, on_leave_sessions, attendance_pct
-> FROM v_attendance_percentage
-> WHERE student_id = 1;

+-----+-----+-----+-----+-----+-----+-----+
| roll_no | student_name | course_code | attended | total_sessions | on_leave_sessions | attendance_pct |
+-----+-----+-----+-----+-----+-----+-----+
| CS101   | Rahul Verma  | CS301       | 2        | 4              | 0                | 50.00         |
| CS101   | Rahul Verma  | CS302       | 1        | 2              | 0                | 50.00         |
+-----+-----+-----+-----+-----+-----+-----+
```

2. Insert: a new leave request (student Rohit Sharma)

INSERT Figure 2. Before: leave_request table in the Database tab

The Database tab reads the table straight from MySQL. leave_request holds 6 rows.

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave

Login as Rohit Sharma (EC301)

My Attendance

Apply Leave

My Leaves

Database

10

88

14

3

2

2

1

tables

rows

foreign keys

indexes

views

triggers

procedure

Everything on this tab is read live from the MySQL database, so it changes as you apply, approve and mark attendance.

Tables

Schema

Views & queries

Activity log

department

5 rows

faculty

7 rows

student

7 rows

course

5 rows

enrollment

11 rows

class_session

9 rows

attendance

26 rows

leave_type

5 rows

leave_request

6 rows

leave_approval

7 rows

leave_request 6 rows

request_id	student_id	type_id	from_date	to_date	reason	status	applied_on
1	3	1	2026-08-24	2026-08-25	Viral fever	Approved	2026-10-06 0
2	4	2	2026-08-31	2026-08-31	Family function	Rejected	2026-10-06 0
3	2	2	2026-09-09	2026-09-10	Cousin wedding	Rejected	2026-10-06 0
4	5	3	2026-09-01	2026-09-03	Inter-college symposium	Approved	2026-10-06 0
5	6	1	2026-09-21	2026-09-22	Dental surgery	Mentor Approved	2026-10-06 0
6	1	1	2026-09-15	2026-09-16	Fever, doctor advised rest	Pending	2026-10-06 0

INSERT Figure 3. Apply Leave form filled in

Medical leave from 12 to 13 October 2026, reason "Family function".

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave

Login as Rohit Sharma (EC301)

My Attendance

Apply Leave

My Leaves

Database

New leave request

Leave type
Medical (10 of 10 days left)

From
10/12/2026

To
10/13/2026

Reason
Family function

Submit request

The stored procedure `sp_apply_leave` in MySQL checks the dates, overlaps and the leave balance.

INSERT Figure 4. After: request submitted

The app confirms request #7 with status Pending, and My Leaves lists it.

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave

Login as Rohit Sharma (EC301)

My Attendance

Apply Leave

My Leaves

Database

My leave requests

#	Type	From	To	Days	Status	Approval trail
7	Medical	2026-10-12	2026-10-13	2	Pending	No decisions yet Cancel

▼ SQL sent to MySQL (last action)

CALL sp_apply_leave(7,1,'2026-10-12','2026-10-13','Family function');

Leave request #7 submitted (status Pending)

INSERT Figure 5. After: the same table read back from MySQL

leave_request now holds 7 rows, so the new record is stored in the database.

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave

Login as Rohit Sharma (EC301)

My Attendance

Apply Leave

My Leaves

Database

10

89

14

3

2

2

1

tables

rows

foreign keys

indexes

views

triggers

procedure

Everything on this tab is read live from the MySQL database, so it changes as you apply, approve and mark attendance.

Tables

Schema

Views & queries

Activity log

department

5 rows

faculty

7 rows

student

7 rows

course

5 rows

enrollment

11 rows

class_session

9 rows

attendance

26 rows

leave_type

5 rows

leave_request

7 rows

leave_approval

7 rows

leave_request 7 rows

request_id	student_id	type_id	from_date	to_date	reason	status	applied_on
1	3	1	2026-08-24	2026-08-25	Viral fever	Approved	2026-10-06 0
2	4	2	2026-08-31	2026-08-31	Family function	Rejected	2026-10-06 0
3	2	2	2026-09-09	2026-09-10	Cousin wedding	Rejected	2026-10-06 0
4	5	3	2026-09-01	2026-09-03	Inter-college symposium	Approved	2026-10-06 0
5	6	1	2026-09-21	2026-09-22	Dental surgery	Mentor Approved	2026-10-06 0
6	1	1	2026-09-15	2026-09-16	Fever, doctor advised rest	Pending	2026-10-06 0
7	7	1	2026-10-12	2026-10-13	Family function	Pending	2026-10-06 0

MYSQL MySQL output for Figure 5

leave_request after the insert: request 7 (Rohit Sharma, 12 to 13 October, Pending).

```
MySQL command line: database attendance_leave

mysql> SELECT request_id, student_id, type_id, from_date, to_date, reason, status
-> FROM leave_request
-> ORDER BY request_id DESC LIMIT 3;
+-----+-----+-----+-----+-----+-----+-----+
| request_id | student_id | type_id | from_date | to_date | reason | status |
+-----+-----+-----+-----+-----+-----+-----+
| 7 | 7 | 1 | 2026-10-12 | 2026-10-13 | Family function | Pending |
| 6 | 1 | 1 | 2026-09-15 | 2026-09-16 | Fever, doctor advised rest | Pending |
| 5 | 6 | 1 | 2026-09-21 | 2026-09-22 | Dental surgery | Mentor Approved |
+-----+-----+-----+-----+-----+-----+-----+
```

RULE Figure 6. Overlapping leave is rejected

Rahul asks for 16 to 17 September, which overlaps his pending request. The stored procedure `sp_apply_leave` refuses it and the message from MySQL is shown.

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave

Login as

Rahul Verma (CS101)

My Attendance

Apply Leave

My Leaves

Database

New leave request

Leave type

Medical (8 of 10 days left)

From

09/16/2026

To

09/17/2026

Reason

Doctor's visit

Submit request

The stored procedure `sp_apply_leave` in MySQL checks the dates, overlaps and the leave balance.

▼ SQL sent to MySQL (last action)

MySQL rejected it: Overlapping leave request exists

CALL sp_apply_leave(1,1,'2026-09-16','2026-09-17','Doctor\'s visit');

Overlapping leave request exists

3. Delete: cancel a pending request

DELETE Figure 7. Before: My Leaves

Request #7 is Pending and has a Cancel button.

 **Attendance & Leave Approval System**

● MySQL 10.11.14 · attendance_leave

Login as

Rohit Sharma (EC301) ▼

My Attendance

Apply Leave

My Leaves

Database

My leave requests

#	Type	From	To	Days	Status	Approval trail
7	Medical	2026-10-12	2026-10-13	2	Pending	No decisions yet Cancel

DELETE Figure 8. After: request cancelled

The status is now Cancelled. The panel shows the UPDATE sent to MySQL. The row is kept so the history stays on record.

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave

Login as Rohit Sharma (EC301)

My Attendance

Apply Leave

My Leaves

Database

My leave requests

#	Type	From	To	Days	Status	Approval trail
7	Medical	2026-10-12	2026-10-13	2	Cancelled	No decisions yet

SQL sent to MySQL (last action)

UPDATE leave_request SET status='Cancelled' WHERE request_id=7 AND student_id=7 AND status IN ('Pending','Mentor Approved')

Request cancelled

MYSQL MySQL output for Figure 8

Request 7 after the cancel: status Cancelled.

```
MySQL command line: database attendance_leave

mysql> SELECT request_id, status
-> FROM leave_request
-> WHERE request_id = 7;
+-----+-----+
| request_id | status |
+-----+-----+
|          7 | Cancelled |
+-----+-----+
```

4. Two-level approval and its effect on attendance

VIEW Figure 9. Leave Approvals as the mentor (Prof. Kiran Das)

Request #6 from Rahul Verma is waiting at the Mentor level. The trail of every request is listed below.

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave

Login as Prof. Kiran Das

Mark Attendance

Leave Approvals

Attendance Report

Records

Database

Awaiting your decision

#	Student	Type	Dates	Days	Reason	Your level	Remarks
6	Rahul Verma	Medical	2026-09-15 → 2026-09-16	2	Fever, doctor advised rest	Mentor	optional Approve Reject

All leave requests

#	Student	Dates	Status	Trail
7	Rohit Sharma	2026-10-12 → 2026-10-13	Cancelled	No decisions yet
6	Rahul Verma	2026-09-15 → 2026-09-16	Pending	No decisions yet
5	Kavya Menon	2026-09-21 → 2026-09-22	Mentor Approved	Mentor: Approved by Prof. Rohan Iyer ("Surgery letter attached")
4	Arjun Mehta	2026-09-01 → 2026-09-03	Approved	Mentor: Approved by Prof. Rohan Iyer ("Department nominee") HOD: Approved by Dr. Meera Nair ("Approved for symposium")
3	Sneha Reddy	2026-09-09 → 2026-09-10	Rejected	Mentor: Rejected by Prof. Kiran Das ("Lab exam scheduled")
2	Priya Nair	2026-08-31 → 2026-08-31	Rejected	Mentor: Rejected by Prof. Neha Joshi ("Internal test that day")
1	Aman Gupta	2026-08-24 → 2026-08-25	Approved	Mentor: Approved by Prof. Neha Joshi ("Medical certificate seen") HOD: Approved by Dr. Anita Rao

INSERT Figure 10. After the mentor approves

Request #6 becomes Mentor Approved. A row was inserted into leave_approval and a trigger moved the request forward.

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave

Login as Prof. Kiran Das

Mark Attendance

Leave Approvals

Attendance Report

Records

Database

Awaiting your decision

Nothing pending for you.

All leave requests

#	Student	Dates	Status	Trail
7	Rohit Sharma	2026-10-12 → 2026-10-13	Cancelled	No decisions yet
6	Rahul Verma	2026-09-15 → 2026-09-16	Mentor Approved	Mentor: Approved by Prof. Kiran Das ("Get well soon")
5	Kavya Menon	2026-09-21 → 2026-09-22	Mentor Approved	Mentor: Approved by Prof. Rohan Iyer ("Surgery letter attached")
4	Arjun Mehta	2026-09-01 → 2026-09-03	Approved	Mentor: Approved by Prof. Rohan Iyer ("Department nominee") HOD: Approved by Dr. Meera Nair ("Approved for symposium")
3	Sneha Reddy	2026-09-09 → 2026-09-10	Rejected	Mentor: Rejected by Prof. Kiran Das ("Lab exam scheduled")
2	Priya Nair	2026-08-31 → 2026-08-31	Rejected	Mentor: Rejected by Prof. Neha Joshi ("Internal test that day")
1	Aman Gupta	2026-08-24 → 2026-08-25	Approved	Mentor: Approved by Prof. Neha Joshi ("Medical certificate seen") HOD: Approved by Dr. Anita Rao

SQL sent to MySQL (last action)

Request #6 is now Mentor Approved

INSERT INTO leave_approval (request_id, level, approver_id, decision, remarks) VALUES (6, 'Mentor', 2, 'Approved', 'Get well s

VIEW Figure 11. Leave Approvals as the HOD (Dr. Anita Rao)

The same request now waits at the HOD level. The HOD cannot decide before the mentor.

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave

Login as Dr. Anita Rao (HOD)

Mark Attendance

Leave Approvals

Attendance Report

Records

Database

Awaiting your decision

#	Student	Type	Dates	Days	Reason	Your level	Remarks
6	Rahul Verma	Medical	2026-09-15 → 2026-09-16	2	Fever, doctor advised rest	HOD	optional Approve Reject

All leave requests

#	Student	Dates	Status	Trail
7	Rohit Sharma	2026-10-12 → 2026-10-13	Cancelled	No decisions yet
6	Rahul Verma	2026-09-15 → 2026-09-16	Mentor Approved	Mentor: Approved by Prof. Kiran Das ("Get well soon")
5	Kavya Menon	2026-09-21 → 2026-09-22	Mentor Approved	Mentor: Approved by Prof. Rohan Iyer ("Surgery letter attached")
4	Arjun Mehta	2026-09-01 → 2026-09-03	Approved	Mentor: Approved by Prof. Rohan Iyer ("Department nominee") HOD: Approved by Dr. Meera Nair ("Approved for symposium")
3	Sneha Reddy	2026-09-09 → 2026-09-10	Rejected	Mentor: Rejected by Prof. Kiran Das ("Lab exam scheduled")
2	Priya Nair	2026-08-31 → 2026-08-31	Rejected	Mentor: Rejected by Prof. Neha Joshi ("Internal test that day")
1	Aman Gupta	2026-08-24 → 2026-08-25	Approved	Mentor: Approved by Prof. Neha Joshi ("Medical certificate seen") HOD: Approved by Dr. Anita Rao

INSERT Figure 12. After the HOD approves

Request #6 is Approved and the trigger sets 3 attendance rows to On Leave (see the message at the bottom).

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave

Login as Dr. Anita Rao (HOD)

Mark Attendance

Leave Approvals

Attendance Report

Records

Database

Awaiting your decision

Nothing pending for you.

All leave requests

#	Student	Dates	Status	Trail
7	Rohit Sharma	2026-10-12 → 2026-10-13	Cancelled	No decisions yet
6	Rahul Verma	2026-09-15 → 2026-09-16	Approved	Mentor: Approved by Prof. Kiran Das ("Get well soon") HOD: Approved by Dr. Anita Rao
5	Kavya Menon	2026-09-21 → 2026-09-22	Mentor Approved	Mentor: Approved by Prof. Rohan Iyer ("Surgery letter attached")
4	Arjun Mehta	2026-09-01 → 2026-09-03	Approved	Mentor: Approved by Prof. Rohan Iyer ("Department nominee") HOD: Approved by Dr. Meera Nair ("Approved for symposium")
3	Sneha Reddy	2026-09-09 → 2026-09-10	Rejected	Mentor: Rejected by Prof. Kiran Das ("Lab exam scheduled")
2	Priya Nair	2026-08-31 → 2026-08-31	Rejected	Mentor: Rejected by Prof. Neha Joshi ("Internal test that day")
1	Aman Gupta	2026-08-24 → 2026-08-25	Approved	Mentor: Approved by Prof. Neha Joshi ("Medical certificate seen") HOD: Approved by Dr. Anita Rao

Request #6 is now Approved – 3 attendance rows set to On Leave

SQL sent to MySQL (last action)

INSERT INTO leave_approval (request_id, level, approver_id, decision, remarks) VALUES (6, 'HOD', 1, 'Approved', NULL);

MYSQL MySQL output for Figure 12

After both approvals: leave_approval has the Mentor and HOD rows, request 6 is Approved, and the trigger set three attendance rows to On Leave.

MySQL command line: database attendance_leave

```
mysql> SELECT request_id, level, approver_id, decision
-> FROM leave_approval
-> WHERE request_id = 6;
```

request_id	level	approver_id	decision
6	Mentor	2	Approved
6	HOD	1	Approved

```
mysql> SELECT request_id, status
-> FROM leave_request
-> WHERE request_id = 6;
```

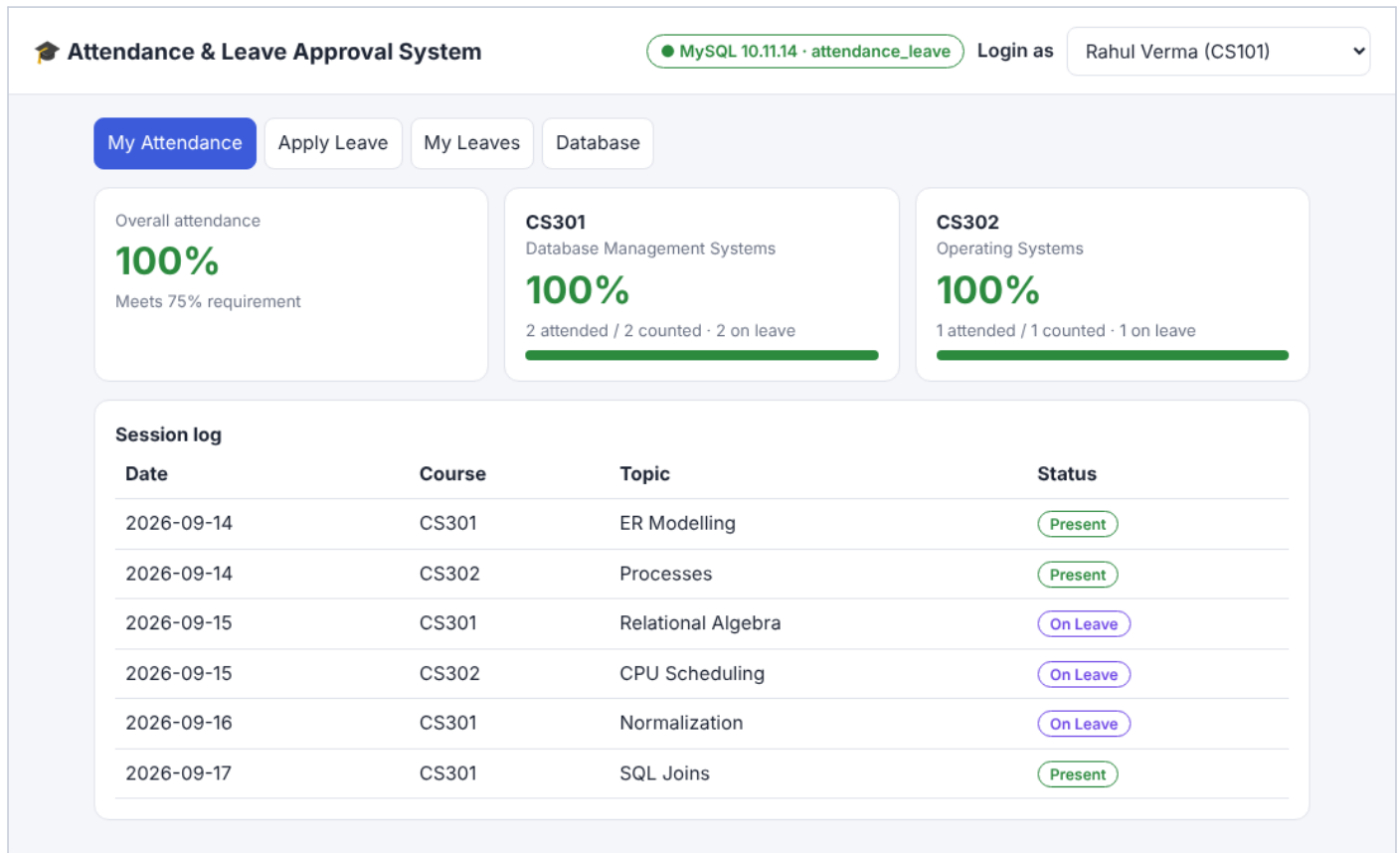
request_id	status
6	Approved

```
mysql> SELECT session_id, student_id, status
-> FROM attendance
-> WHERE student_id = 1 AND status = 'On Leave';
```

session_id	student_id	status
2	1	On Leave
3	1	On Leave
6	1	On Leave

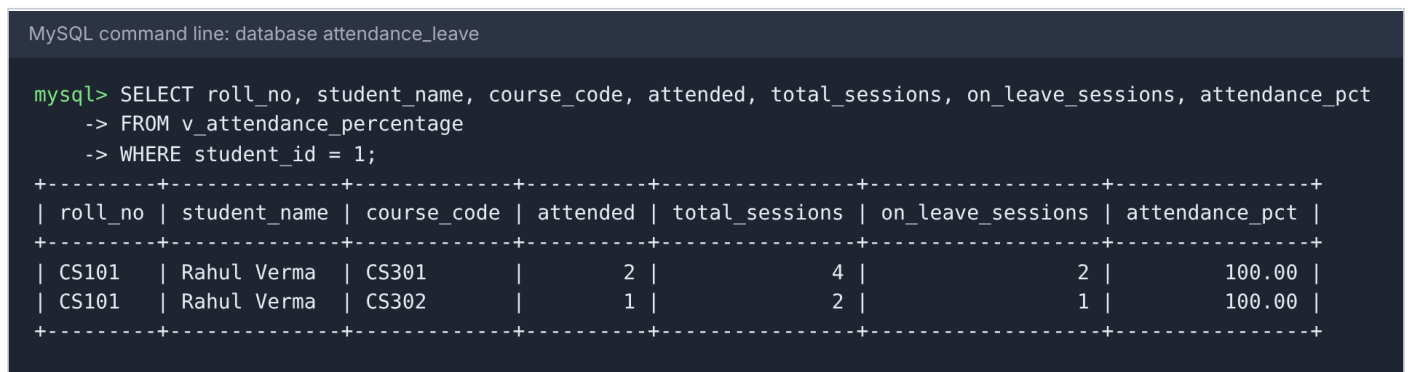
VIEW Figure 13. Rahul's My Attendance after approval

Overall attendance is now 100%. The absences inside the approved leave dates count as On Leave and are left out of the percentage.



MYSQL MySQL output for Figure 13

After approval: 100% in both courses, with the On Leave sessions left out of the count.



5. Insert and delete a record: department

INSERT Figure 14. Before: Records tab, department table

The table has 5 rows. The form is generated from the table definition.

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave

Login as Dr. Anita Rao (HOD)

Mark Attendance

Leave Approvals

Attendance Report

Records

Database

department

faculty

student

course

enrollment

class_session

attendance

leave_type

leave_request

leave_approval

Insert into department

dept_name varchar(60)

Insert record

department 5 rows

dept_id PK	dept_name	
1	Computer Science	Delete
2	Information Technology	Delete
3	Electronics	Delete
4	Mechanical	Delete
5	Business Administration	Delete

INSERT Figure 15. Insert form filled in

New department "Civil Engineering".

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave

Login as Dr. Anita Rao (HOD)

Mark Attendance

Leave Approvals

Attendance Report

Records

Database

department

faculty

student

course

enrollment

class_session

attendance

leave_type

leave_request

leave_approval

Insert into department

dept_name varchar(60)

Civil Engineering

Insert record

department 5 rows

dept_id	dept_name	
1	Computer Science	Delete
2	Information Technology	Delete
3	Electronics	Delete
4	Mechanical	Delete
5	Business Administration	Delete

INSERT Figure 16. After: record inserted

The table has 6 rows. The panel shows the INSERT statement sent to MySQL.

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave

Login as Dr. Anita Rao (HOD)

Mark Attendance

Leave Approvals

Attendance Report

Records

Database

department

faculty

student

course

enrollment

class_session

attendance

leave_type

leave_request

leave_approval

Insert into department

dept_name varchar(60)

Insert record

department 6 rows

dept_id (PK)	dept_name	
1	Computer Science	Delete
2	Information Technology	Delete
3	Electronics	Delete
4	Mechanical	Delete
5	Business Administration	Delete
6	Civil Engineering	Delete

1 row inserted into department

▼ SQL sent to MySQL (last action)

INSERT INTO `department` (`dept\_name`) VALUES ('Civil Engineering');

MySQL MySQL output for Figure 16

department after the insert: 6 rows, including Civil Engineering with dept_id 6.

MySQL command line: database attendance_leave

```
mysql> SELECT *
-> FROM department;
+-----+-----+
| dept_id | dept_name          |
+-----+-----+
| 5       | Business Administration |
| 6       | Civil Engineering    |
| 1       | Computer Science     |
| 3       | Electronics          |
| 2       | Information Technology |
| 4       | Mechanical           |
+-----+-----+
```

INSERT Figure 17. After: the table read back in the Database tab
 department shows 6 rows including Civil Engineering.

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave
 Login as
 Dr. Anita Rao (HOD)

Mark Attendance
 Leave Approvals
 Attendance Report
 Records
 Database

10

92

14

3

2

2

1

tables

rows

foreign keys

indexes

views

triggers

procedure

Everything on this tab is read live from the MySQL database, so it changes as you apply, approve and mark attendance.

Tables

Schema

Views & queries

Activity log

department

6 rows

faculty

7 rows

student

7 rows

course

5 rows

enrollment

11 rows

class_session

9 rows

attendance

26 rows

leave_type

5 rows

leave_request

7 rows

leave_approval

9 rows

department 6 rows

dept_id PK	dept_name
1	Computer Science
2	Information Technology
3	Electronics
4	Mechanical
5	Business Administration
6	Civil Engineering

DELETE Figure 18. Before: delete

Civil Engineering is in the table.

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave

Login as Dr. Anita Rao (HOD)

Mark Attendance

Leave Approvals

Attendance Report

Records

Database

department

faculty

student

course

enrollment

class_session

attendance

leave_type

leave_request

leave_approval

Insert into department

dept_name varchar(60)

Insert record

department 6 rows

dept_id (PK)	dept_name	
1	Computer Science	Delete
2	Information Technology	Delete
3	Electronics	Delete
4	Mechanical	Delete
5	Business Administration	Delete
6	Civil Engineering	Delete

DELETE Figure 19. Delete asks for confirmation

The first click turns the button into "Confirm?". A second click runs the delete.

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave

Login as Dr. Anita Rao (HOD)

Mark Attendance

Leave Approvals

Attendance Report

Records

Database

department

faculty

student

course

enrollment

class_session

attendance

leave_type

leave_request

leave_approval

Insert into department

dept_name varchar(60)

Insert record

department 6 rows

dept_id (PK)	dept_name	
1	Computer Science	Delete
2	Information Technology	Delete
3	Electronics	Delete
4	Mechanical	Delete
5	Business Administration	Delete
6	Civil Engineering	Confirm?

DELETE Figure 20. After: record deleted

The table is back to 5 rows. The panel shows the DELETE statement sent to MySQL.

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave

Login as Dr. Anita Rao (HOD)

Mark Attendance

Leave Approvals

Attendance Report

Records

Database

department

faculty

student

course

enrollment

class_session

attendance

leave_type

leave_request

leave_approval

Insert into department

dept_name varchar(60)

Insert record

department 5 rows

dept_id (PK)	dept_name	
1	Computer Science	Delete
2	Information Technology	Delete
3	Electronics	Delete
4	Mechanical	Delete
5	Business Administration	Delete

▼ SQL sent to MySQL (last action)

1 row deleted from department

DELETE FROM `department` WHERE `dept\_id`=6;

MYSQL MySQL output for Figure 20

department after the delete: back to 5 rows.

```
MySQL command line: database attendance_leave

mysql> SELECT *
-> FROM department;

+-----+-----+
| dept_id | dept_name |
+-----+-----+
| 5 | Business Administration |
| 1 | Computer Science |
| 3 | Electronics |
| 2 | Information Technology |
| 4 | Mechanical |
+-----+-----+
```

RULE Figure 21. Deleting a department in use is refused

Computer Science is still used by courses, so MySQL rejects the delete with a foreign key constraint error and nothing changes.

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave

Login as Dr. Anita Rao (HOD)

Mark Attendance

Leave Approvals

Attendance Report

Records

Database

department

faculty

student

course

enrollment

class_session

attendance

leave_type

leave_request

leave_approval

Insert into department

dept_name varchar(60)

Insert record

department 5 rows

dept_id (PK)	dept_name	
1	Computer Science	Confirm?
2	Information Technology	Delete
3	Electronics	Delete
4	Mechanical	Delete
5	Business Administration	Delete

SQL sent to MySQL (last

Cannot delete or update a parent row: a foreign key constraint fails ('attendance_leave'. 'course', CONSTRAINT 'fk_course_dept' FOREIGN KEY ('dept_id') REFERENCES 'department' ('dept_id'))

MySQL rejected it: Cannot delete or update a parent row: a foreign key constraint fails ('attendance_leave'. 'course', CONSTRAINT 'fk_course_dept' FOREIGN KEY ('dept_id') REFERENCES 'department' ('dept_id'))

DELETE FROM `department` WHERE `dept\_id`=1;

6. Insert and delete a record: student

INSERT Figure 22. Before: Records tab, student table

The table has 7 rows. Mentor and department are dropdowns, because they are foreign keys.

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave

Login as Dr. Anita Rao (HOD)

Mark Attendance

Leave Approvals

Attendance Report

Records

Database

department

faculty

student

course

enrollment

class_session

attendance

leave_type

leave_request

leave_approval

Insert into student

roll_no varchar(20)

name varchar(80)

semester tinyint(4)

mentor_id int(11) FK

Select...

dept_id int(11) FK

Select...

Insert record

student 7 rows

student_id PK	roll_no	name	semester	mentor_id FK	dept_id FK	
1	CS101	Rahul Verma	5	2	1	Delete
2	CS102	Sneha Reddy	5	2	1	Delete
3	CS103	Aman Gupta	5	6	1	Delete
4	CS104	Priya Nair	5	6	1	Delete
5	IT201	Arjun Mehta	5	4	2	Delete
6	IT202	Kavya Menon	5	4	2	Delete
7	EC301	Rohit Sharma	5	7	3	Delete

INSERT Figure 23. Insert form filled in

Roll number CS199, name Demo Student, semester 5.

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave

Login as Dr. Anita Rao (HOD)

Mark Attendance

Leave Approvals

Attendance Report

Records

Database

department

faculty

student

course

enrollment

class_session

attendance

leave_type

leave_request

leave_approval

Insert into student

roll_no varchar(20)

name varchar(80)

semester tinyint(4)

mentor_id int(11) (FK)

CS199

Demo Student

5

1 · Dr. Anita Rao

dept_id int(11) (FK)

1 · Computer Science

Insert record

student 7 rows

student_id (PK)	roll_no	name	semester	mentor_id (FK)	dept_id (FK)	
1	CS101	Rahul Verma	5	2	1	Delete
2	CS102	Sneha Reddy	5	2	1	Delete
3	CS103	Aman Gupta	5	6	1	Delete
4	CS104	Priya Nair	5	6	1	Delete
5	IT201	Arjun Mehta	5	4	2	Delete
6	IT202	Kavya Menon	5	4	2	Delete
7	EC301	Rohit Sharma	5	7	3	Delete

INSERT Figure 24. After: record inserted

The table has 8 rows.

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave

Login as Dr. Anita Rao (HOD)

Mark Attendance

Leave Approvals

Attendance Report

Records

Database

department

faculty

student

course

enrollment

class_session

attendance

leave_type

leave_request

leave_approval

Insert into student

roll_no varchar(20)

name varchar(80)

semester tinyint(4)

mentor_id int(11) (FK)

Select...

dept_id int(11) (FK)

Select...

Insert record

student 8 rows

student_id (PK)	roll_no	name	semester	mentor_id (FK)	dept_id (FK)	
1	CS101	Rahul Verma	5	2	1	Delete
2	CS102	Sneha Reddy	5	2	1	Delete
3	CS103	Aman Gupta	5	6	1	Delete
4	CS104	Priya Nair	5	6	1	Delete
5	IT201	Arjun Mehta			2	Delete
6	IT202	Kavya Menon	5	4	2	Delete
7	EC301	Rohit Sharma	5	7	3	Delete
8	CS199	Demo Student	5	1	1	Delete

1 row inserted into student

SQL sent to MySQL (last action)

INSERT INTO `student` (`roll\_no`, `name`, `semester`, `mentor\_id`, `dept\_id`) VALUES ('CS199', 'Demo Student', '5', '1', '1')

MYSQL MySQL output for Figure 24

student after the insert: CS199 Demo Student as student_id 8.

MySQL command line: database attendance_leave

```
mysql> SELECT student_id, roll_no, name, semester, mentor_id, dept_id  
-> FROM student  
-> ORDER BY student_id DESC LIMIT 3;
```

student_id	roll_no	name	semester	mentor_id	dept_id
8	CS199	Demo Student	5	1	1
7	EC301	Rohit Sharma	5	7	3
6	IT202	Kavya Menon	5	4	2

RULE Figure 25. Invalid value is rejected

Semester 12 breaks the CHECK constraint chk_semester (1 to 8), so MySQL refuses the insert.

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave

Login as Dr. Anita Rao (HOD)

Mark Attendance

Leave Approvals

Attendance Report

Records

Database

department

faculty

student

course

enrollment

class_session

attendance

leave_type

leave_request

leave_approval

Insert into student

roll_no varchar(20)
CS198

name varchar(80)
Bad Semester

semester tinyint(4)
12

mentor_id int(11) FK
1 · Dr. Anita Rao

dept_id int(11) FK
1 · Computer Science

Insert record

student 8 rows

student_id PK	roll_no	name	semester	mentor_id FK	dept_id FK	
1	CS101	Rahul Verma	5	2	1	Delete
2	CS102	Sneha Reddy	5	2	1	Delete
3	CS103	Aman Gupta	5	6	1	Delete
4	CS104	Priya Nair	5	6	1	Delete
5	IT20	CONSTRAINT 'chk_semester' failed for 'attendance_leave'. 'student'				Delete
6	IT202	Kavya Menon	5	4	2	Delete
7	EC301	Rohit Sharma	5	7	3	Delete
8	CS199	Demo Student	5	1	1	Delete

SQL sent to MySQL (last action)

MySQL rejected it: CONSTRAINT 'chk_semester' failed for 'attendance_leave'. 'student'

INSERT INTO `student` (`roll\_no`, `name`, `semester`, `mentor\_id`, `dept\_id`) VALUES ('CS198', 'Bad Semester', '12', '1',

DELETE Figure 26. After: record deleted

CS199 is removed and the table is back to 7 rows.

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave

Login as Dr. Anita Rao (HOD)

Mark Attendance

Leave Approvals

Attendance Report

Records

Database

department

faculty

student

course

enrollment

class_session

attendance

leave_type

leave_request

leave_approval

Insert into student

roll_no varchar(20)

name varchar(80)

semester tinyint(4)

mentor_id int(11) (FK)

dept_id int(11) (FK)

Insert record

student 7 rows

student_id (PK)	roll_no	name	semester	mentor_id (FK)	dept_id (FK)	
1	CS101	Rahul Verma	5	2	1	Delete
2	CS102	Sneha Reddy	5	2	1	Delete
3	CS103	Aman Gupta	5	6	1	Delete
4	CS104	Priya Nair	5	6	1	Delete
5	IT201	Arjun Mehta	5	4	2	Delete
6	IT202	Kavya Menon	5	4	2	Delete
7	EC301	Rohit Sharma	5	7	3	Delete

SQL sent to MySQL (last action)

DELETE FROM `student` WHERE `student\_id`=8;

MYSQL MySQL output for Figure 26

student after the delete: CS199 is gone.

```
MySQL command line: database attendance_leave

mysql> SELECT student_id, roll_no, name
-> FROM student
-> ORDER BY student_id;
+-----+-----+-----+
| student_id | roll_no | name |
+-----+-----+-----+
| 1 | CS101 | Rahul Verma |
| 2 | CS102 | Sneha Reddy |
| 3 | CS103 | Aman Gupta |
| 4 | CS104 | Priya Nair |
| 5 | IT201 | Arjun Mehta |
| 6 | IT202 | Kavya Menon |
| 7 | EC301 | Rohit Sharma |
+-----+-----+-----+
```

7. Attendance: marking and the report

VIEW Figure 27. Mark Attendance (Prof. Kiran Das)

Class session: 14 September, CS301, ER Modelling. Every student starts as Present.

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave

Login as Prof. Kiran Das

Mark Attendance

Leave Approvals

Attendance Report

Records

Database

Mark attendance

Class session

2026-09-14 – CS301 – ER Modelling

Roll	Student	Status	
CS101	Rahul Verma	Present	saved
CS102	Sneha Reddy	Present	saved
CS103	Aman Gupta	Present	saved
CS104	Priya Nair	Present	saved

Save attendance

INSERT Figure 28. After: attendance saved

Aman Gupta is changed to Late and saved. The panel shows the statements sent to MySQL.

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave

Login as Prof. Kiran Das

Mark Attendance

Leave Approvals

Attendance Report

Records

Database

Mark attendance

Class session

2026-09-14 – CS301 – ER Modelling

Roll	Student	Status	
CS101	Rahul Verma	Present	saved
CS102	Sneha Reddy	Present	saved
CS103	Aman Gupta	Late	saved
CS104	Priya Nair	Present	saved

Save attendance

SQL sent to MySQL (last action)

INSERT INTO attendance (session_id, student_id, status) VALUES (1,1,'Present') ON DUPLICATE KEY UPDATE status=VALUES(status);
INSERT INTO attendance (session_id, student_id, status) VALUES (1,2,'Present') ON DUPLICATE KEY UPDATE status=VALUES(status);
INSERT INTO attendance (session_id, student_id, status) VALUES (1,3,'Late') ON DUPLICATE KEY UPDATE status=VALUES(status);
INSERT INTO attendance (session_id, student_id, status) VALUES (1,4,'Present') ON DUPLICATE KEY UPDATE status=VALUES(status);

Attendance saved (4 rows)

MYSQL MySQL output for Figure 28

attendance for session 1 (ER Modelling): student 3, Aman Gupta, is Late.

```
MySQL command line: database attendance_leave

mysql> SELECT session_id, student_id, status
-> FROM attendance
-> WHERE session_id = 1;
+-----+-----+-----+
| session_id | student_id | status |
+-----+-----+-----+
|          1 |          1 | Present |
|          1 |          2 | Present |
|          1 |          3 | Late    |
|          1 |          4 | Present |
+-----+-----+-----+
```

VIEW Figure 29. Attendance report

Built from the view `v_attendance_percentage`. Values below 75% are shown in red (Sneha Reddy in CS302, Kavya Menon in IT301).

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave

Login as Prof. Kiran Das

Mark Attendance

Leave Approvals

Attendance Report

Records

Database

Attendance report (shortage below 75% highlighted)

From the view `v_attendance_percentage`.

Roll	Student	CS301	CS302	CS303	EC301	IT301	Overall
CS101	Rahul Verma	100%	100%	–	–	–	100%
CS102	Sneha Reddy	100%	50%	–	–	–	83.33%
CS103	Aman Gupta	75%	100%	–	–	–	83.33%
CS104	Priya Nair	100%	–	100%	–	–	100%
EC301	Rohit Sharma	–	–	–	100%	–	100%
IT201	Arjun Mehta	–	–	–	–	100%	100%
IT202	Kavya Menon	–	–	–	–	0%	0%

MYSQL MySQL output for Figure 29

Shortage list: students below 75% (Kavya Menon in IT301, Sneha Reddy in CS302).

MySQL command line: database attendance_leave

```
mysql> SELECT roll_no, student_name, course_code, attendance_pct
-> FROM v_attendance_percentage
-> WHERE attendance_pct < 75
-> ORDER BY attendance_pct;
```

```
+-----+-----+-----+-----+
| roll_no | student_name | course_code | attendance_pct |
+-----+-----+-----+-----+
| IT202   | Kavya Menon  | IT301       | 0.00          |
| CS102   | Sneha Reddy  | CS302       | 50.00         |
+-----+-----+-----+-----+
```

8. Database tab: what is stored in MySQL

VIEW Figure 30. Tables

Row counts for all ten tables, read live from MySQL.

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave

Login as Prof. Kiran Das

Mark Attendance

Leave Approvals

Attendance Report

Records

Database

10

91

14

3

2

2

1

tables

rows

foreign keys

indexes

views

triggers

procedure

Everything on this tab is read live from the MySQL database, so it changes as you apply, approve and mark attendance.

Tables

Schema

Views & queries

Activity log

department

5 rows

faculty

7 rows

student

7 rows

course

5 rows

enrollment

11 rows

class_session

9 rows

attendance

26 rows

leave_type

5 rows

leave_request

7 rows

leave_approval

9 rows

department 5 rows

dept_id (PK)	dept_name
1	Computer Science
2	Information Technology
3	Electronics
4	Mechanical
5	Business Administration

VIEW Figure 31. Schema (part 1 of 4)

The real CREATE TABLE, view, stored procedure and trigger definitions from MySQL.

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave

Login as Prof. Kiran Das

Mark Attendance

Leave Approvals

Attendance Report

Records

Database

10

91

14

3

2

2

1

tables

rows

foreign keys

indexes

views

triggers

procedure

Everything on this tab is read live from the MySQL database, so it changes as you apply, approve and mark attendance.

Tables

Schema

Views & queries

Activity log

Tables (SHOW CREATE TABLE)

CREATE TABLE `department` (
 `dept\_id` int(11) NOT NULL AUTO_INCREMENT,
 `dept\_name` varchar(60) NOT NULL,
 PRIMARY KEY (`dept\_id`),
 UNIQUE KEY `dept\_name` (`dept\_name`)
) ENGINE=InnoDB AUTO_INCREMENT=7 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci

CREATE TABLE `faculty` (
 `faculty\_id` int(11) NOT NULL AUTO_INCREMENT,
 `name` varchar(80) NOT NULL,
 `email` varchar(100) NOT NULL,
 `is\_hod` tinyint(1) NOT NULL DEFAULT 0,
 `dept\_id` int(11) NOT NULL,
 PRIMARY KEY (`faculty\_id`),
 UNIQUE KEY `email` (`email`),
 KEY `fk\_faculty\_dept` (`dept\_id`),
 CONSTRAINT `fk\_faculty\_dept` FOREIGN KEY (`dept\_id`) REFERENCES `department` (`dept\_id`)
) ENGINE=InnoDB AUTO_INCREMENT=8 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci

CREATE TABLE `student` (
 `student\_id` int(11) NOT NULL AUTO_INCREMENT,
 `roll\_no` varchar(20) NOT NULL,
 `name` varchar(80) NOT NULL,
 `semester` tinyint(4) NOT NULL,
 `mentor\_id` int(11) NOT NULL,
 `dept\_id` int(11) NOT NULL,
 PRIMARY KEY (`student\_id`),
 UNIQUE KEY `roll\_no` (`roll\_no`),
 KEY `fk\_student\_mentor` (`mentor\_id`),
 KEY `fk\_student\_dept` (`dept\_id`),
 CONSTRAINT `fk\_student\_dept` FOREIGN KEY (`dept\_id`) REFERENCES `department` (`dept\_id`),
 CONSTRAINT `fk\_student\_mentor` FOREIGN KEY (`mentor\_id`) REFERENCES `faculty` (`faculty\_id`),
 CONSTRAINT `chk\_semester` CHECK (`semester` between 1 and 8)
) ENGINE=InnoDB AUTO_INCREMENT=9 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci

CREATE TABLE `course` (
 `course\_id` int(11) NOT NULL AUTO_INCREMENT,
 `course\_code` varchar(10) NOT NULL,
 `course\_name` varchar(80) NOT NULL,
 `credits` tinyint(4) NOT NULL,
 `dept\_id` int(11) NOT NULL,
 `faculty\_id` int(11) NOT NULL,
 PRIMARY KEY (`course\_id`),

Student Attendance and Leave Approval System: UI screenshots

Page 35

VIEW Figure 31. Schema (part 2 of 4)

```
UNIQUE KEY `course_code` (`course_code`),
KEY `fk_course_dept` (`dept_id`),
KEY `fk_course_faculty` (`faculty_id`),
CONSTRAINT `fk_course_dept` FOREIGN KEY (`dept_id`) REFERENCES `department` (`dept_id`),
CONSTRAINT `fk_course_faculty` FOREIGN KEY (`faculty_id`) REFERENCES `faculty` (`faculty_id`),
CONSTRAINT `chk_credits` CHECK (`credits` between 1 and 6)
) ENGINE=InnoDB AUTO_INCREMENT=6 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
```

```
CREATE TABLE `enrollment` (
  `student_id` int(11) NOT NULL,
  `course_id` int(11) NOT NULL,
  `academic_year` char(7) NOT NULL,
  PRIMARY KEY (`student_id`,`course_id`,`academic_year`),
  KEY `fk_enr_course` (`course_id`),
  CONSTRAINT `fk_enr_course` FOREIGN KEY (`course_id`) REFERENCES `course` (`course_id`) ON DELETE CASCADE,
  CONSTRAINT `fk_enr_student` FOREIGN KEY (`student_id`) REFERENCES `student` (`student_id`) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
```

```
CREATE TABLE `class_session` (
  `session_id` int(11) NOT NULL AUTO_INCREMENT,
  `course_id` int(11) NOT NULL,
  `session_date` date NOT NULL,
  `start_time` time NOT NULL,
  `end_time` time NOT NULL,
  `topic` varchar(100) NOT NULL,
  PRIMARY KEY (`session_id`),
  KEY `fk_session_course` (`course_id`),
  KEY `idx_session_date` (`session_date`),
  CONSTRAINT `fk_session_course` FOREIGN KEY (`course_id`) REFERENCES `course` (`course_id`) ON DELETE CASCADE,
  CONSTRAINT `chk_session_time` CHECK (`end_time` > `start_time`)
) ENGINE=InnoDB AUTO_INCREMENT=10 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
```

```
CREATE TABLE `attendance` (
  `session_id` int(11) NOT NULL,
  `student_id` int(11) NOT NULL,
  `status` enum('Present','Absent','Late','On Leave') NOT NULL,
  PRIMARY KEY (`session_id`,`student_id`),
  KEY `idx_attendance_student` (`student_id`),
  CONSTRAINT `fk_att_session` FOREIGN KEY (`session_id`) REFERENCES `class_session` (`session_id`) ON DELETE CASCADE,
  CONSTRAINT `fk_att_student` FOREIGN KEY (`student_id`) REFERENCES `student` (`student_id`) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
```

```
CREATE TABLE `leave_type` (
  `type_id` int(11) NOT NULL AUTO_INCREMENT,
  `type_name` varchar(40) NOT NULL,
  `max_days_per_sem` int(11) NOT NULL,
  PRIMARY KEY (`type_id`),
  UNIQUE KEY `type_name` (`type_name`),
  CONSTRAINT `chk_max_days` CHECK (`max_days_per_sem` > 0)
) ENGINE=InnoDB AUTO_INCREMENT=6 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
```

```
CREATE TABLE `leave_request` (
  `request_id` int(11) NOT NULL AUTO_INCREMENT,
  `student_id` int(11) NOT NULL,
  `type_id` int(11) NOT NULL,
  `from_date` date NOT NULL,
  `to_date` date NOT NULL,
  `reason` varchar(255) NOT NULL,
  `status` enum('Pending','Mentor Approved','Approved','Rejected','Cancelled') NOT NULL DEFAULT 'Pending',
  `applied_on` datetime NOT NULL DEFAULT current_timestamp(),
  PRIMARY KEY (`request_id`),
```

VIEW Figure 31. Schema (part 3 of 4)

```
KEY `fk_leave_type` (`type_id`),
KEY `idx_leave_student_status` (`student_id`, `status`),
CONSTRAINT `fk_leave_student` FOREIGN KEY (`student_id`) REFERENCES `student` (`student_id`) ON DELETE CASCADE,
CONSTRAINT `fk_leave_type` FOREIGN KEY (`type_id`) REFERENCES `leave_type` (`type_id`),
CONSTRAINT `chk_leave_dates` CHECK (`to_date` >= `from_date`)
) ENGINE=InnoDB AUTO_INCREMENT=8 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
```

```
CREATE TABLE `leave_approval` (
  `request_id` int(11) NOT NULL,
  `level` enum('Mentor','HOD') NOT NULL,
  `approver_id` int(11) NOT NULL,
  `decision` enum('Approved','Rejected') NOT NULL,
  `remarks` varchar(255) DEFAULT NULL,
  `decided_on` datetime NOT NULL DEFAULT current_timestamp(),
  PRIMARY KEY (`request_id`, `level`),
  KEY `fk_appr_approver` (`approver_id`),
  CONSTRAINT `fk_appr_approver` FOREIGN KEY (`approver_id`) REFERENCES `faculty` (`faculty_id`),
  CONSTRAINT `fk_appr_request` FOREIGN KEY (`request_id`) REFERENCES `leave_request` (`request_id`) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
```

Views

```
CREATE ALGORITHM=UNDEFINED SQL SECURITY DEFINER VIEW `v_attendance_percentage` AS select `s`.`student_id` AS `student_id`,
```

```
CREATE ALGORITHM=UNDEFINED SQL SECURITY DEFINER VIEW `v_pending_leaves` AS select `lr`.`request_id` AS `request_id`,`s`.`r
```

Stored procedure

```
CREATE PROCEDURE `sp_apply_leave`(
  IN p_student_id INT, IN p_type_id INT,
  IN p_from DATE, IN p_to DATE, IN p_reason VARCHAR(255))
BEGIN
  DECLARE v_max      INT;
  DECLARE v_used      INT;
  DECLARE v_requested INT;

  IF p_to < p_from THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'To-date cannot be before from-date';
  END IF;

  IF EXISTS (SELECT 1 FROM leave_request
             WHERE student_id = p_student_id
                   AND status IN ('Pending', 'Mentor Approved', 'Approved')
                   AND from_date <= p_to AND to_date >= p_from) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Overlapping leave request exists';
  END IF;

  SELECT max_days_per_sem INTO v_max FROM leave_type WHERE type_id = p_type_id;
  IF v_max IS NULL THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Unknown leave type';
  END IF;

  SET v_requested = DATEDIFF(p_to, p_from) + 1;
  SELECT COALESCE(SUM(DATEDIFF(to_date, from_date) + 1), 0) INTO v_used
  FROM leave_request
  WHERE student_id = p_student_id AND type_id = p_type_id
        AND status IN ('Pending', 'Mentor Approved', 'Approved');

  IF v_used + v_requested > v_max THEN
```

VIEW Figure 31. Schema (part 4 of 4)

```
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Leave balance exceeded for this leave type';
END IF;

INSERT INTO leave_request (student_id, type_id, from_date, to_date, reason)
VALUES (p_student_id, p_type_id, p_from, p_to, p_reason);

SELECT LAST_INSERT_ID() AS request_id;
END
```

Triggers

```
CREATE TRIGGER trg_approval_before_insert
BEFORE INSERT ON leave_approval
FOR EACH ROW
BEGIN
    DECLARE v_status VARCHAR(20);
    SELECT status INTO v_status FROM leave_request WHERE request_id = NEW.request_id;

    IF NEW.level = 'Mentor' THEN
        IF v_status <> 'Pending' THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Mentor can decide only on a Pending request';
        END IF;
        IF NOT EXISTS (SELECT 1 FROM leave_request lr JOIN student s ON s.student_id = lr.student_id
                        WHERE lr.request_id = NEW.request_id AND s.mentor_id = NEW.approver_id) THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Only the student''s own mentor can give the Mentor decision';
        END IF;
    ELSE
        IF v_status <> 'Mentor Approved' THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'HOD can decide only after mentor approval';
        END IF;
        IF NOT EXISTS (SELECT 1 FROM leave_request lr
                        JOIN student s ON s.student_id = lr.student_id
                        JOIN faculty f ON f.faculty_id = NEW.approver_id
                        WHERE lr.request_id = NEW.request_id AND f.is_hod = 1 AND f.dept_id = s.dept_id) THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Only the HOD of the student''s department can give the HOD decision';
        END IF;
    END IF;
END
```

```
CREATE TRIGGER trg_approval_after_insert
AFTER INSERT ON leave_approval
FOR EACH ROW
BEGIN
    IF NEW.decision = 'Rejected' THEN
        UPDATE leave_request SET status = 'Rejected' WHERE request_id = NEW.request_id;
    ELSEIF NEW.level = 'Mentor' THEN
        UPDATE leave_request SET status = 'Mentor Approved' WHERE request_id = NEW.request_id;
    ELSE
        UPDATE leave_request SET status = 'Approved' WHERE request_id = NEW.request_id;

        UPDATE attendance a
        JOIN class_session cs ON cs.session_id = a.session_id
        JOIN leave_request lr ON lr.request_id = NEW.request_id
        SET a.status = 'On Leave'
        WHERE a.student_id = lr.student_id
        AND a.status = 'Absent'
        AND cs.session_date BETWEEN lr.from_date AND lr.to_date;
    END IF;
END
```

VIEW Figure 32. Views and report queries (part 1 of 5)

Both views and the nine report queries Q1 to Q9 with their live results.

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave

Login as Prof. Kiran Das

Mark Attendance

Leave Approvals

Attendance Report

Records

Database

10

91

14

3

2

2

1

tables

rows

foreign keys

indexes

views

triggers

procedure

Everything on this tab is read live from the MySQL database, so it changes as you apply, approve and mark attendance.

Tables

Schema

Views & queries

Activity log

VIEWS

v_attendance_percentage

SELECT * FROM v_attendance_percentage; (definition on the Schema tab)

student_id	roll_no	student_name	dept_id	course_id	course_code	course_name	total_sessions	on_
1	CS101	Rahul Verma	1	1	CS301	Database Management Systems	4	2
1	CS101	Rahul Verma	1	2	CS302	Operating Systems	2	1
2	CS102	Sneha Reddy	1	1	CS301	Database Management Systems	4	0
2	CS102	Sneha Reddy	1	2	CS302	Operating Systems	2	0
3	CS103	Aman Gupta	1	1	CS301	Database Management Systems	4	0
3	CS103	Aman Gupta	1	2	CS302	Operating Systems	2	0
4	CS104	Priya Nair	1	1	CS301	Database Management Systems	4	0
4	CS104	Priya Nair	1	3	CS303	Computer Networks	1	0
5	IT201	Arjun Mehta	2	4	IT301	Web Technologies	1	0
6	IT202	Kavya Menon	2	4	IT301	Web Technologies	1	0
7	EC301	Rohit Sharma	3	5	EC301	Digital Signal Processing	1	0

11 rows in set

v_pending_leaves

SELECT * FROM v_pending_leaves; (definition on the Schema tab)

request_id	roll_no	student_name	type_name	from_date	to_date	days	awaiting	mentor_name
5	IT202	Kavya Menon	Medical	2026-09-21	2026-09-22	2	HOD	Prof. Rohan Iyer

1 row in set

REPORT QUERIES

Q1: Attendance percentage per student and course

VIEW Figure 32. Views and report queries (part 2 of 5)

Present + Late count as attended; On Leave sessions are removed from the total.

```
SELECT roll_no, student_name, course_code, attended,
       total_sessions - on_leave_sessions AS counted,
       attendance_pct,
       CASE WHEN attendance_pct < 75 THEN 'SHORTAGE' ELSE 'OK' END AS flag
FROM v_attendance_percentage
ORDER BY roll_no, course_code;
```

roll_no	student_name	course_code	attended	counted	attendance_pct	flag
CS101	Rahul Verma	CS301	2	2	100%	OK
CS101	Rahul Verma	CS302	1	1	100%	OK
CS102	Sneha Reddy	CS301	4	4	100%	OK
CS102	Sneha Reddy	CS302	1	2	50%	SHORTAGE
CS103	Aman Gupta	CS301	3	4	75%	OK
CS103	Aman Gupta	CS302	2	2	100%	OK
CS104	Priya Nair	CS301	4	4	100%	OK
CS104	Priya Nair	CS303	1	1	100%	OK
EC301	Rohit Sharma	EC301	1	1	100%	OK
IT201	Arjun Mehta	IT301	1	1	100%	OK
IT202	Kavya Menon	IT301	0	1	0%	SHORTAGE

11 rows in set

Q2: Shortage list, students below 75% in a course

Joins the view to the student's mentor so the right person can follow up.

```
SELECT v.roll_no, v.student_name, v.course_code, v.attendance_pct, m.name AS mentor
FROM v_attendance_percentage v
JOIN student s ON s.student_id = v.student_id
JOIN faculty m ON m.faculty_id = s.mentor_id
WHERE v.attendance_pct < 75
ORDER BY v.attendance_pct, v.roll_no;
```

roll_no	student_name	course_code	attendance_pct	mentor
IT202	Kavya Menon	IT301	0%	Prof. Rohan Iyer
CS102	Sneha Reddy	CS302	50%	Prof. Kiran Das

2 rows in set

Q3: Approval trail of every leave request

One row per decision, Mentor first and HOD second.

```
SELECT lr.request_id, s.name AS student, la.level, f.name AS approver,
       la.decision, la.remarks, lr.status AS request_status
FROM leave_approval la
```

VIEW Figure 32. Views and report queries (part 3 of 5)

```
JOIN leave_request lr ON lr.request_id = la.request_id
JOIN student s      ON s.student_id = lr.student_id
JOIN faculty f      ON f.faculty_id = la.approver_id
ORDER BY lr.request_id, la.level;
```

request_id	student	level	approver	decision	remarks	request_status
1	Aman Gupta	Mentor	Prof. Neha Joshi	Approved	Medical certificate seen	Approved
1	Aman Gupta	HOD	Dr. Anita Rao	Approved	NULL	Approved
2	Priya Nair	Mentor	Prof. Neha Joshi	Rejected	Internal test that day	Rejected
3	Sneha Reddy	Mentor	Prof. Kiran Das	Rejected	Lab exam scheduled	Rejected
4	Arjun Mehta	Mentor	Prof. Rohan Iyer	Approved	Department nominee	Approved
4	Arjun Mehta	HOD	Dr. Meera Nair	Approved	Approved for symposium	Approved
5	Kavya Menon	Mentor	Prof. Rohan Iyer	Approved	Surgery letter attached	Mentor Approved
6	Rahul Verma	Mentor	Prof. Kiran Das	Approved	Get well soon	Approved
6	Rahul Verma	HOD	Dr. Anita Rao	Approved	NULL	Approved

9 rows in set

Q4: Leave days used against the limit, per student and leave type

Pending, Mentor Approved and Approved requests count against the balance.

```
SELECT s.name AS student, lt.type_name,
       SUM(DATEDIFF(lr.to_date, lr.from_date) + 1) AS days_used,
       lt.max_days_per_sem,
       lt.max_days_per_sem - SUM(DATEDIFF(lr.to_date, lr.from_date) + 1) AS days_left
FROM leave_request lr
JOIN student s      ON s.student_id = lr.student_id
JOIN leave_type lt  ON lt.type_id = lr.type_id
WHERE lr.status IN ('Pending', 'Mentor Approved', 'Approved')
GROUP BY s.student_id, s.name, lt.type_id, lt.type_name, lt.max_days_per_sem
ORDER BY s.name, lt.type_name;
```

student	type_name	days_used	max_days_per_sem	days_left
Aman Gupta	Medical	2	10	8
Arjun Mehta	Academic Event	3	7	4
Kavya Menon	Medical	2	10	8
Rahul Verma	Medical	2	10	8

4 rows in set

Q5: Pending requests per mentor

LEFT JOIN keeps mentors with nothing pending (count 0).

```
SELECT m.name AS mentor, COUNT(lr.request_id) AS pending_requests
FROM faculty m
JOIN student s ON s.mentor_id = m.faculty_id
LEFT JOIN leave_request lr ON lr.student_id = s.student_id AND lr.status = 'Pending'
```

VIEW Figure 32. Views and report queries (part 4 of 5)

```
GROUP BY m.faculty_id, m.name
ORDER BY pending_requests DESC, m.name;
```

mentor	pending_requests
Prof. Kiran Das	0
Prof. Neha Joshi	0
Prof. Rohan Iyer	0
Prof. Vikram Singh	0

4 rows in set

Q6: Requests waiting for the HOD

The mentor has approved; this shows which HOD has to decide.

```
SELECT lr.request_id, s.name AS student, lt.type_name, lr.from_date, lr.to_date,
       h.name AS hod_to_decide
FROM leave_request lr
JOIN student s      ON s.student_id = lr.student_id
JOIN leave_type lt  ON lt.type_id = lr.type_id
JOIN faculty h      ON h.dept_id = s.dept_id AND h.is_hod = 1
WHERE lr.status = 'Mentor Approved'
ORDER BY lr.from_date;
```

request_id	student	type_name	from_date	to_date	hod_to_decide
5	Kavya Menon	Medical	2026-09-21	2026-09-22	Dr. Meera Nair

1 row in set

Q7: Course-wise average attendance and number of students in shortage

```
SELECT course_code, course_name,
       COUNT(*) AS students,
       ROUND(AVG(attendance_pct), 2) AS avg_attendance_pct,
       SUM(attendance_pct < 75) AS in_shortage
FROM v_attendance_percentage
GROUP BY course_id, course_code, course_name
ORDER BY avg_attendance_pct;
```

course_code	course_name	students	avg_attendance_pct	in_shortage
IT301	Web Technologies	2	50%	1
CS302	Operating Systems	3	83.33%	1
CS301	Database Management Systems	4	93.75%	0
EC301	Digital Signal Processing	1	100%	0
CS303	Computer Networks	1	100%	0

5 rows in set

VIEW Figure 32. Views and report queries (part 5 of 5)

Q8: Session-wise headcount by attendance status

```
SELECT cs.session_date, c.course_code, cs.topic,
       SUM(a.status = 'Present') AS present,
       SUM(a.status = 'Late')    AS late,
       SUM(a.status = 'Absent')  AS absent,
       SUM(a.status = 'On Leave') AS on_leave
FROM class_session cs
JOIN course c ON c.course_id = cs.course_id
LEFT JOIN attendance a ON a.session_id = cs.session_id
GROUP BY cs.session_id, cs.session_date, c.course_code, cs.topic
ORDER BY cs.session_date, c.course_code;
```

session_date	course_code	topic	present	late	absent	on_leave
2026-09-14	CS301	ER Modelling	3	1	0	0
2026-09-14	CS302	Processes	2	0	1	0
2026-09-15	CS301	Relational Algebra	2	1	0	1
2026-09-15	CS302	CPU Scheduling	2	0	0	1
2026-09-15	IT301	HTML and CSS	1	0	1	0
2026-09-16	CS301	Normalization	2	0	1	1
2026-09-16	CS303	OSI Model	1	0	0	0
2026-09-16	EC301	Sampling Theorem	1	0	0	0
2026-09-17	CS301	SQL Joins	3	1	0	0

9 rows in set

Q9: Leave requests per department and status

```
SELECT d.dept_name, lr.status,
       COUNT(*) AS requests,
       SUM(DATEDIFF(lr.to_date, lr.from_date) + 1) AS total_days
FROM leave_request lr
JOIN student s ON s.student_id = lr.student_id
JOIN department d ON d.dept_id = s.dept_id
GROUP BY d.dept_id, d.dept_name, lr.status
ORDER BY d.dept_name, lr.status;
```

dept_name	status	requests	total_days
Computer Science	Approved	2	4
Computer Science	Rejected	2	3
Electronics	Cancelled	1	2
Information Technology	Mentor Approved	1	2
Information Technology	Approved	1	3

5 rows in set

VIEW Figure 33. Activity log (part 1 of 2)

Every write sent to MySQL by the UI, in order.

Attendance & Leave Approval System

MySQL 10.11.14 · attendance_leave

Login as Prof. Kiran Das

Mark Attendance

Leave Approvals

Attendance Report

Records

Database

10

91

14

3

2

2

1

tables

rows

foreign keys

indexes

views

triggers

procedure

Everything on this tab is read live from the MySQL database, so it changes as you apply, approve and mark attendance.

Tables

Schema

Views & queries

Activity log

Every write the app has sent to MySQL since the server started, newest first.

08:01:32 · Prof. Kiran Das

INSERT INTO attendance (session_id, student_id, status) VALUES (1,4,'Present') ON DUPLICATE KEY UPDATE status=VALUES(status)

08:01:32 · Prof. Kiran Das

INSERT INTO attendance (session_id, student_id, status) VALUES (1,3,'Late') ON DUPLICATE KEY UPDATE status=VALUES(status);

08:01:32 · Prof. Kiran Das

INSERT INTO attendance (session_id, student_id, status) VALUES (1,2,'Present') ON DUPLICATE KEY UPDATE status=VALUES(status)

08:01:32 · Prof. Kiran Das

INSERT INTO attendance (session_id, student_id, status) VALUES (1,1,'Present') ON DUPLICATE KEY UPDATE status=VALUES(status)

08:01:30 · Dr. Anita Rao

DELETE FROM `student` WHERE `student\_id`=8;

08:01:29 · Dr. Anita Rao

INSERT INTO `student` (`roll\_no`, `name`, `semester`, `mentor\_id`, `dept\_id`) VALUES ('CS198', 'Bad Semester', '12', '1',

VIEW Figure 33. Activity log (part 2 of 2)

08:01:28 · Dr. Anita Rao	INSERT INTO `student` (`roll_no`, `name`, `semester`, `mentor_id`, `dept_id`) VALUES ('CS199', 'Demo Student', '5', '1', '1');
08:01:27 · Dr. Anita Rao	DELETE FROM `department` WHERE `dept_id`=1;
08:01:26 · Dr. Anita Rao	DELETE FROM `department` WHERE `dept_id`=6;
08:01:24 · Dr. Anita Rao	INSERT INTO `department` (`dept_name`) VALUES ('Civil Engineering');
08:01:20 · Dr. Anita Rao	INSERT INTO leave_approval (request_id, level, approver_id, decision, remarks) VALUES (6, 'HOD', 1, 'Approved', NULL);
08:01:19 · Prof. Kiran Das	INSERT INTO leave_approval (request_id, level, approver_id, decision, remarks) VALUES (6, 'Mentor', 2, 'Approved', 'Get well s');
08:01:18 · Rahul Verma	CALL sp_apply_leave(1, 1, '2026-09-16', '2026-09-17', 'Doctor\'s visit');
08:01:16 · Rohit Sharma	UPDATE leave_request SET status='Cancelled' WHERE request_id=7 AND student_id=7 AND status IN ('Pending', 'Mentor Approved');
08:01:14 · Rohit Sharma	CALL sp_apply_leave(7, 1, '2026-10-12', '2026-10-13', 'Family function');

VIEW Figure 34. Mobile layout, dark mode

The same app on a phone-sized screen.

